Chương 2. Các mô hình dữ liệu

Mô hình thực thể - liên kết (tiếp): tinh chỉnh lược đồ và ER mở rộng

Đỗ Thanh Hà

Khoa Trí tuệ nhân tạo
Học viện Công nghệ Bưu chính Viễn thông
Posts and Telecommunications Institute of Technology

Nội dung

1. Mô hình ER: thực thể, thuộc tính, liên kết, khóa (bài trước).
2. Ảnh hưởng của tỉ lệ lực lượng lên khóa của tập liên kết.
3. Đặt vị trí cho thuộc tính của liên kết.
4. Tinh chỉnh lược đồ: thực thể hay thuộc tính; thực thể mạnh hay yếu.
5. Mô hình ER mở rộng (EER): cụ thể hóa, tổng quát hóa, kế thừa và các ràng buộc.
6. Kết hợp (aggregation) và liên kết nhiều ngôi.
7. Đối chiếu lược đồ ER với sơ đồ lớp UML.
8. Ràng buộc toàn vẹn tham chiếu.
9. Các nguyên tắc thiết kế mức khái niệm.

Mục tiêu bài học

- Xác định khóa chính của một tập liên kết theo tỉ lệ lực lượng của nó.
- Quyết định đặt một thuộc tính vào tập thực thể hay vào tập liên kết.
- Quyết định mô hình hóa một khái niệm thành thực thể, thuộc tính hay liên kết.
- Phân biệt thực thể mạnh / thực thể yếu và xác định khóa của thực thể yếu.
- Áp dụng cụ thể hóa, tổng quát hóa cùng ba loại ràng buộc đi kèm.
- Dùng kết hợp (aggregation) để mô hình hóa liên kết giữa các liên kết.

Quy ước thuật ngữ dùng trong bài

Phân biệt hai từ

- **Liên kết** (relationship): khái niệm của **mô hình ER**, vẽ bằng hình thoi.
- **Quan hệ** (relation): khái niệm của **mô hình quan hệ** (Chương 3), là một bảng.

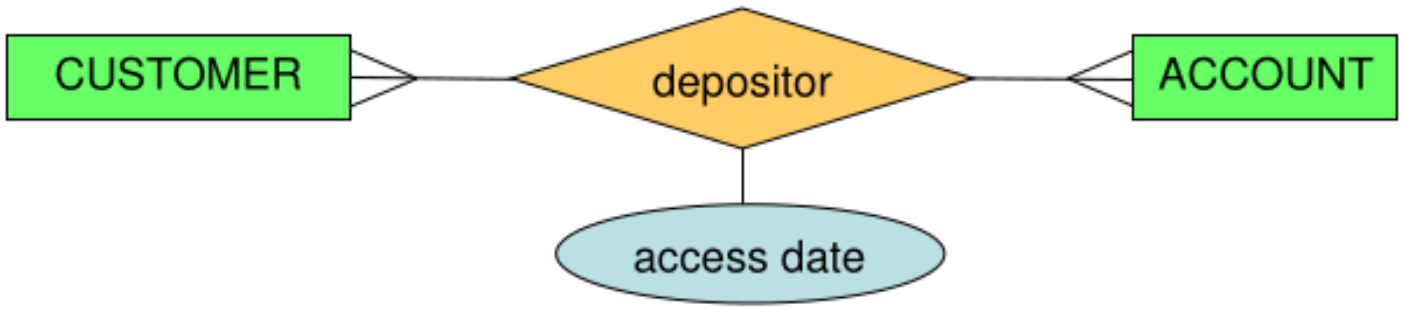
- Trong bài này, mọi khái niệm của ER đều gọi là **liên kết** / **tập liên kết** / **kiểu liên kết**.
- **Lược đồ** của một tập liên kết được viết ở dạng `TEN(thuộc tính,...)` chỉ nhằm minh họa; việc chuyển sang bảng thực sự sẽ được thực hiện ở Chương 3.

PHẦN 1. TỈ LỆ LỰC LƯỢNG VÀ KHÓA CỦA TẬP LIÊN KẾT

Mapping Cardinalities and Keys

Ảnh hưởng của tỉ lệ lực lượng lên khóa

- Khóa chính của một tập liên kết **phụ thuộc vào tỉ lệ lực lượng** của liên kết đó. Xét lược đồ ER sau:



Lược đồ thể hiện liên kết **depositor** có tỉ lệ **M:N** giữa hai tập thực thể **CUSTOMER** và **ACCOUNT**, với thuộc tính riêng **access-date**.

$\Longrightarrow$ Khóa chính của tập liên kết này là **hợp các khóa chính** của hai tập thực thể.

Trường hợp M:N

- Cho hai tập thực thể:

```
CUSTOMER (customer-id, customer-name, address, city)
ACCOUNT (account-number, balance)
```

- **M:N** nghĩa là: một khách hàng có thể truy nhập nhiều tài khoản, và một tài khoản có thể được truy nhập bởi nhiều khách hàng.
- Để biết **lần truy nhập gần nhất của một khách hàng cụ thể tới một tài khoản cụ thể**, cần cả hai khóa để định danh.

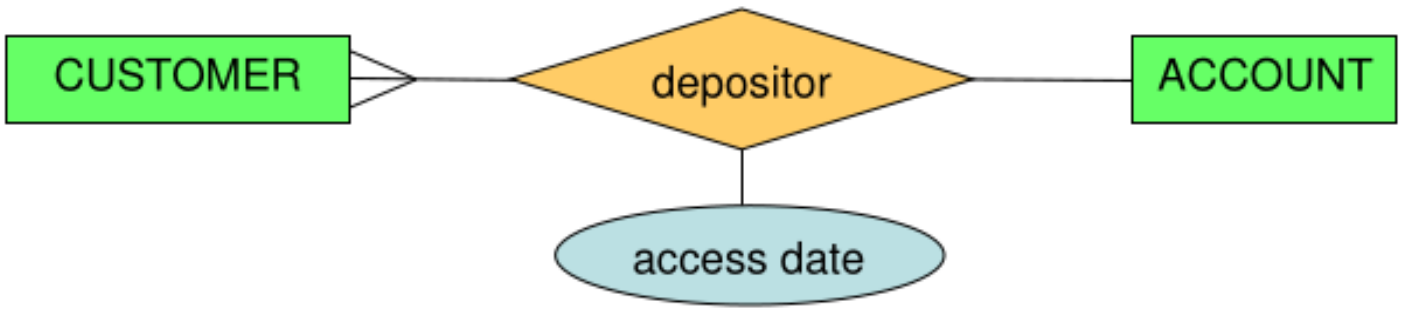
Lược đồ của tập liên kết depositor

```
DEPOSITOR (customer-id, account-number, access-date)
```

Trường hợp M:1 từ CUSTOMER tới ACCOUNT

- Giả sử **mỗi khách hàng chỉ được nộp tiền vào nhiều nhất một tài khoản**, nhưng một tài khoản có thể nhận tiền từ nhiều khách hàng.

  $\Longrightarrow$ **depositor** là liên kết **M:1** từ **CUSTOMER** tới **ACCOUNT**.



  $\Longrightarrow$ Mỗi khách hàng chỉ tham gia nhiều nhất một liên kết $\Rightarrow$ `customer-id` một mình đã định danh duy nhất một liên kết.

Trường hợp M:1 từ CUSTOMER tới ACCOUNT (tiếp)

- Khóa chính của **depositor** chỉ gồm khóa chính của **CUSTOMER**: một khách hàng xác định chỉ có **một** lần nộp tiền gần nhất, tới **một** tài khoản duy nhất mà họ truy nhập được.

Lược đồ của tập liên kết depositor

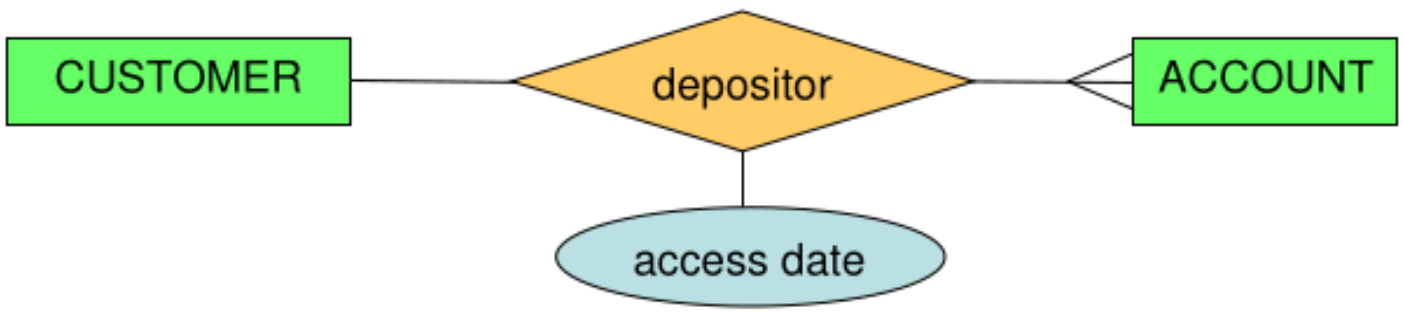
`DEPOSITOR (customer-id, account-number, access-date)`

Chú ý: `account-number` **vẫn phải có mặt** trong lược đồ – nếu bỏ đi thì không còn biết khách hàng đã nộp tiền vào tài khoản nào. Nó chỉ **không thuộc khóa chính**, vì đã được xác định duy nhất qua `customer-id`.

Trường hợp M:1 từ ACCOUNT tới CUSTOMER

- Giả sử **mỗi tài khoản được làm chủ bởi nhiều nhất một khách hàng**, nhưng một khách hàng có thể có nhiều tài khoản.

  $\Longrightarrow$ **depositor** là liên kết **M:1** từ **ACCOUNT** tới **CUSTOMER**.



Trường hợp M:1 từ ACCOUNT tới CUSTOMER (tiếp)

- Mỗi tài khoản chỉ có nhiều nhất một lần nộp tiền gần nhất, và chỉ nhiều nhất một khách hàng có thể thực hiện việc đó.
  $\Longrightarrow$ Khóa chính của **depositor** chỉ gồm khóa chính của **ACCOUNT**.

Lược đồ của tập liên kết depositor

`DEPOSITOR (account-number, customer-id, access-date)`

Quy tắc tổng quát

Lược đồ của tập liên kết **luôn** chứa hợp các khóa chính của các tập thực thể tham gia, cộng các thuộc tính riêng. Chỉ có **khóa chính** là thay đổi theo tỉ lệ lực lượng:

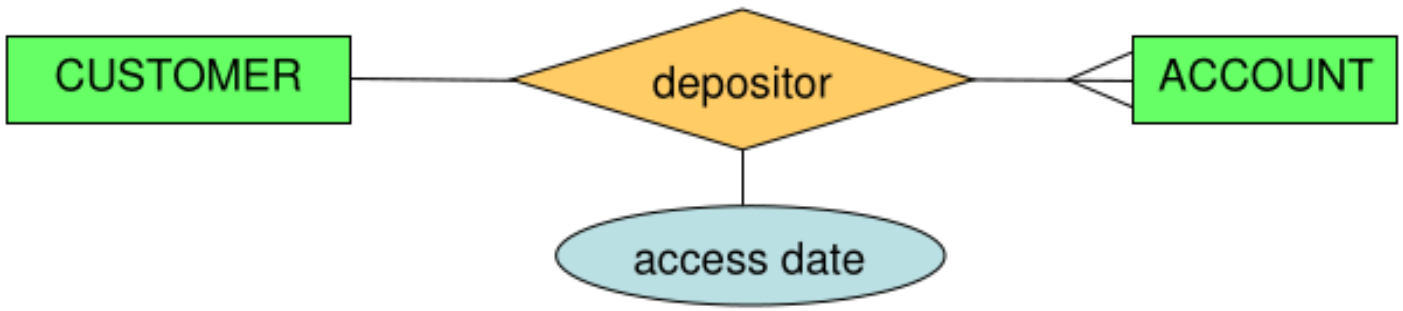
- **M:N** $\rightarrow$ khóa chính $= K_1 \cup K_2$;
- **1:N** $\rightarrow$ khóa chính = khóa của tập thực thể ở phía "nhiều";
- **1:1** $\rightarrow$ K_1 và K_2 đều là khóa dự bị, chọn một làm khóa chính.

PHẦN 2. ĐẶT VỊ TRÍ CHO THUỘC TÍNH CỦA LIÊN KẾT

Placement of Relationship Attributes

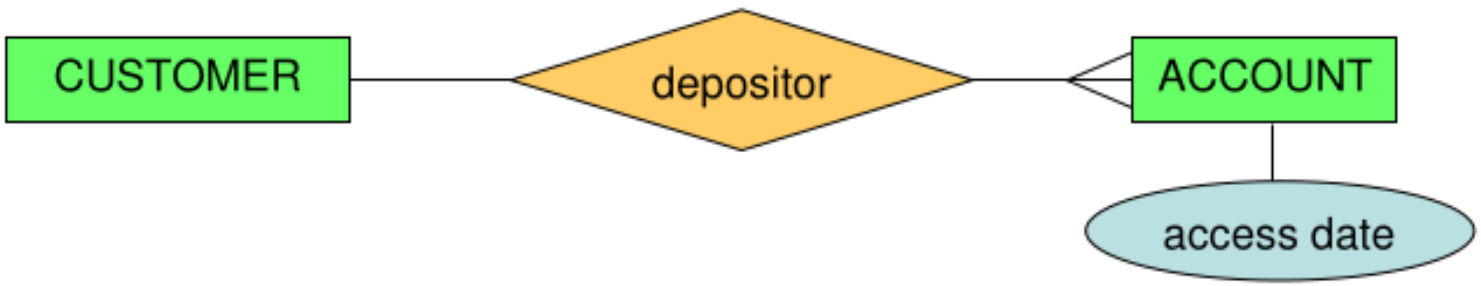
Đặt thuộc tính vào đâu?

- Thuộc tính của một tập liên kết **1:1** hoặc **1:N** thường **có thể chuyển** vào một trong các tập thực thể tham gia, thay vì để ở tập liên kết.
- Với liên kết 1:N, thuộc tính phải chuyển sang tập thực thể ở phía "nhiều" (N) – vì mỗi thực thể ở phía đó chỉ tham gia nhiều nhất một liên kết, nên chỉ nhận một giá trị.
- Ví dụ: liên kết **depositor** với thuộc tính **access-date**.



Chuyển access-date sang ACCOUNT

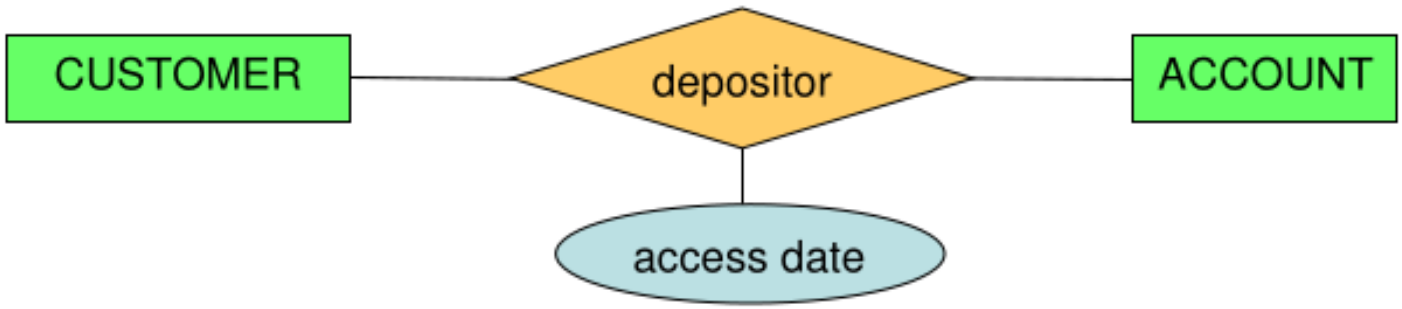
- Với liên kết M:1 từ **ACCOUNT** tới **CUSTOMER**, thuộc tính **access-date** có thể gắn vào **ACCOUNT** mà **không mất thông tin**:



$\Longrightarrow$ Vì mỗi tài khoản thuộc sở hữu của nhiều nhất một khách hàng, nên mỗi tài khoản chỉ có nhiều nhất một giá trị **access-date**.

Trường hợp liên kết 1:1

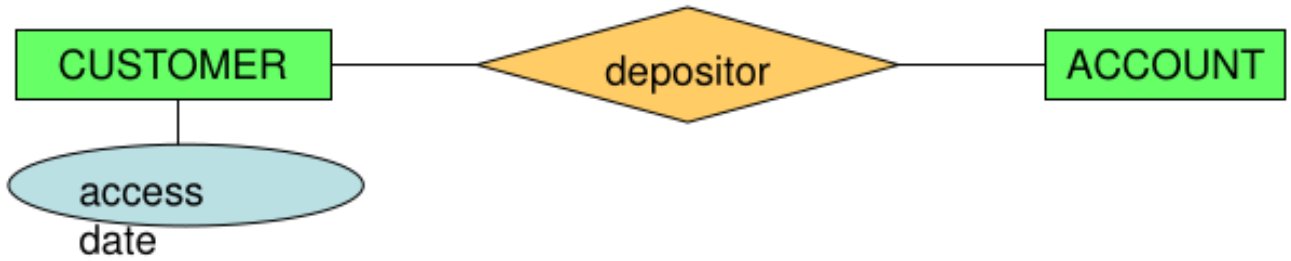
- Xét trường hợp: mỗi tài khoản thuộc nhiều nhất một khách hàng **và** mỗi khách hàng sở hữu nhiều nhất một tài khoản $\Rightarrow$ liên kết **1:1**.



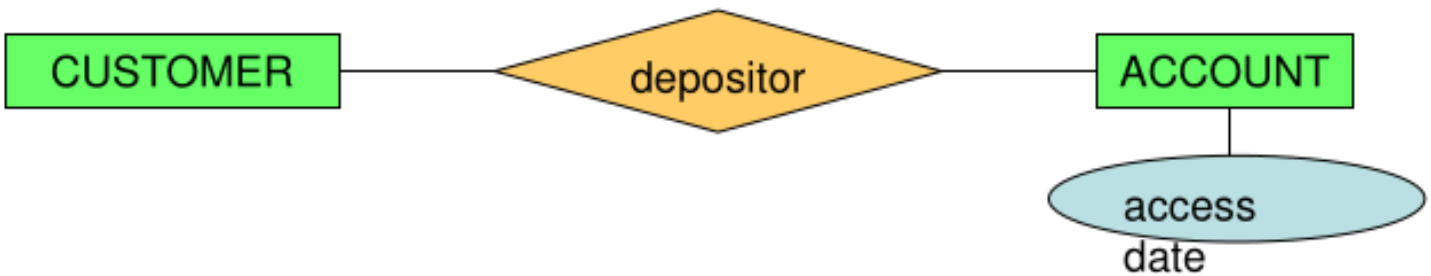
$\Longrightarrow$ Thuộc tính **access-date** có thể gắn vào **CUSTOMER** *hoặc* **ACCOUNT**, đều không mất thông tin. Việc chọn phía nào là quyết định thiết kế, thường dựa trên cách truy vấn hay dùng nhất.

Trường hợp 1:1 – hai cách đặt tương đương

- Đặt **access-date** ở **CUSTOMER**: giá trị này là lần truy nhập cuối của khách hàng tới tài khoản duy nhất mà họ có.

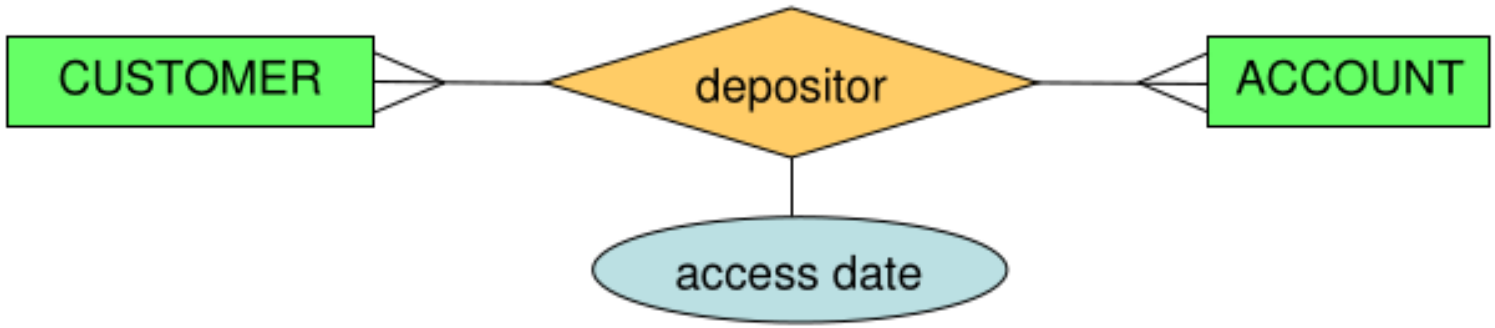


- Đặt **access-date** ở **ACCOUNT**: giá trị này là lần truy nhập cuối tới tài khoản, bởi người khách hàng duy nhất sở hữu nó.



Trường hợp liên kết M:N

- Xét trường hợp: liên kết **depositor** có tỉ lệ **M:N**.



$\Longrightarrow$ Gắn **access-date** vào bất kỳ tập thực thể tham gia nào cũng **làm mất thông tin**, vì mỗi thực thể ở cả hai phía đều có thể tham gia nhiều liên kết.

Kết luận về vị trí thuộc tính

- Nếu cần lưu ngày truy nhập cuối của **một khách hàng cụ thể** tới **một tài khoản cụ thể** thì **access-date nhất thiết** phải là thuộc tính của tập liên kết **depositor**.
  - Nếu để ở **ACCOUNT**: không biết khách hàng nào đã thực hiện giao dịch đó.
  - Nếu để ở **CUSTOMER**: không biết khách hàng đã truy nhập tài khoản nào.

Quy tắc

Một thuộc tính chỉ được chuyển từ tập liên kết sang tập thực thể khi thực thể đó tham gia **nhiều nhất một** liên kết. Ngược lại, thuộc tính **phải** nằm ở tập liên kết.

Bài tập

- Xem lại lược đồ ER các em đã vẽ cho bài toán COMPANY ở buổi trước và kiểm tra:
  1. Mỗi tập liên kết đã ghi đúng tỉ lệ lực lượng và ràng buộc tham gia chưa?
  2. Với từng thuộc tính đặt trên hình thoi: nó **bắt buộc** phải ở đó, hay có thể chuyển về một tập thực thể?
  3. Thuộc tính *số giờ/tuần* của liên kết THAMGIA thuộc loại nào? Vì sao?

PHẦN 3. TINH CHỈNH LƯỢC ĐỒ

Entity vs. Attribute, Strong vs. Weak

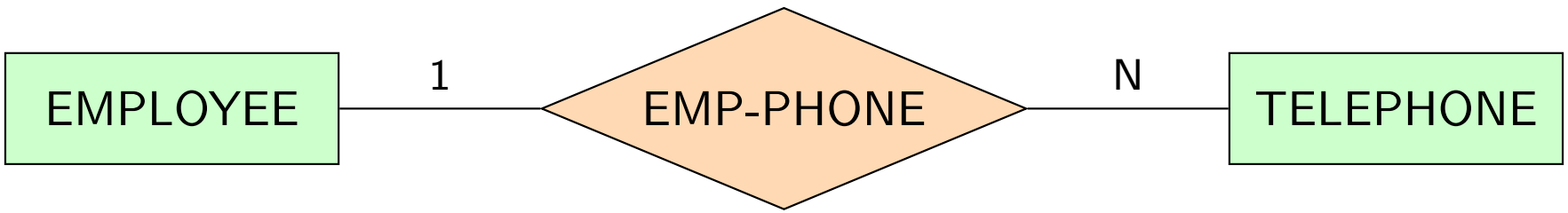
Thực thể hay thuộc tính?

- Xét tập thực thể: `EMPLOYEE(emp-name, telephone-number, age)`
- Nếu máy điện thoại được coi là một **thực thể** (có các thuộc tính riêng *telephone-number, location, manufacturer, serial-num*,...) thì phải tách lược đồ:
  1. Định nghĩa lại: `EMPLOYEE (emp-name, age)`
  2. Tạo tập thực thể mới: `TELEPHONE (telephone-number, location, manufacturer, serial-num, ...)`
  3. Tạo tập liên kết nối nhân viên với máy điện thoại họ sử dụng: `EMP-PHONE (emp-name, telephone-number)`

Lưu ý: lược đồ của tập liên kết chỉ chứa khóa của các tập thực thể tham gia (và thuộc tính riêng của liên kết, nếu có).

Thực thể hay thuộc tính? – tiêu chí

- Coi điện thoại là **thuộc tính** khi: mỗi nhân viên có **một** số điện thoại và ta **không quan tâm** tới thông tin nào khác về máy điện thoại.
- Coi điện thoại là **thực thể** khi có ít nhất một trong các dấu hiệu:
  - máy điện thoại có **thuộc tính riêng** cần lưu (vị trí, hãng sản xuất, số sê-ri. . .);
  - một nhân viên có thể dùng **nhiều** máy;
  - máy điện thoại **tồn tại độc lập** với nhân viên (có thể chưa gán cho ai).



1 nhân viên – nhiều điện thoại (1:N)

$\Longrightarrow$ Vai trò của điện thoại (thực thể hay thuộc tính) phụ thuộc vào ngữ nghĩa của bài toán, không phải vào bản thân dữ liệu.

Liên kết hay thực thể?

- Xét ví dụ ngân hàng: ở bài trước, khoản vay được mô hình hóa như một **thực thể** (**LOAN**).

  Cách khác: mô hình hóa khoản vay như một **liên kết** giữa khách hàng (**CUSTOMER**) và chi nhánh (**BRANCH**), với **loan-number** và **amount** là thuộc tính riêng của liên kết.

  (Chỉ hợp lệ khi: mỗi khoản vay thuộc sở hữu của duy nhất một khách hàng và gắn với duy nhất một chi nhánh.)

Liên kết hay thực thể? – vì sao nên chọn thực thể

- Cách mô hình hóa trên hỏng ngay khi **nhiều khách hàng cùng đứng tên một khoản vay**.

  $\Longrightarrow$ Khi đó phải tạo một liên kết riêng cho **từng** người đồng sở hữu, và **lặp lại** giá trị **loan-number**, **amount** ở mọi liên kết đó.
- Việc lặp giá trị sinh ra hai vấn đề:
  1. Dữ liệu được lưu trữ ở nhiều nơi (dư thừa).
  2. Cập nhật dữ liệu làm tăng nguy cơ **không nhất quán** (sửa chỗ này quên chỗ kia).

  $\Longrightarrow$ Nên giữ LOAN là một tập thực thể riêng.

Thực thể mạnh hay thực thể yếu?

- Tập thực thể **không đủ thuộc tính để tạo khóa chính** gọi là tập thực thể yếu. Tập thực thể có khóa chính riêng gọi là tập thực thể mạnh.
- Ví dụ: tập thực thể các lần trả nợ

  `PAYMENT (payment-number, payment-date, payment-amount)`

  $\Longrightarrow$ **payment-number** thường là số thứ tự bắt đầu từ 1, sinh **riêng rẽ cho từng khoản nợ**. Vì vậy hai lần trả tiền của hai khoản nợ khác nhau có thể trùng **payment-number**.

  $\Longrightarrow$ PAYMENT tự nó không có khóa chính $\Rightarrow$ là tập thực thể yếu.

Thực thể yếu – thực thể định danh và liên kết định danh

- Để có ý nghĩa, tập thực thể yếu phải gắn với một tập thực thể khác, gọi là tập thực thể định danh (hay tập thực thể **sở hữu**).
- Liên kết giữa hai tập này gọi là liên kết định danh (identifying relationship), vẽ bằng **hình thoi nét đôi**.
- Liên kết định danh luôn là liên kết **M:1** từ tập thực thể yếu tới tập thực thể định danh, và sự tham gia của tập thực thể yếu là **đầy đủ (total)**.

  Vì mỗi thực thể yếu phải phụ thuộc vào đúng một thực thể định danh, và không thực thể yếu nào tồn tại mà không có chủ.

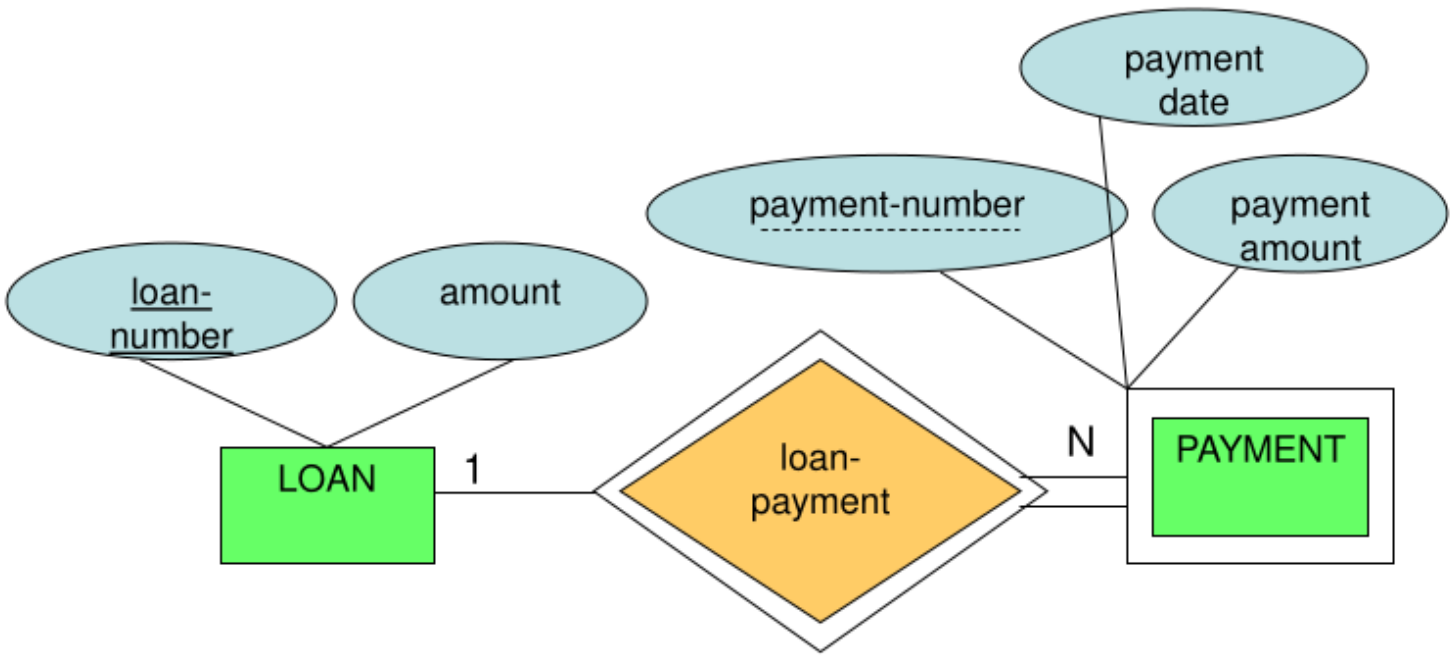
Thực thể yếu – khóa bộ phận

- Tập thực thể yếu không có khóa chính riêng, nhưng vẫn phân biệt được các thực thể của nó **trong phạm vi một thực thể định danh cụ thể**.

  $\Longrightarrow$ Tập thuộc tính làm được việc đó gọi là khóa bộ phận (partial key), còn gọi là **thuộc tính phân biệt** (discriminator). Ký pháp ERD: **gạch chân nét đứt**.
- Ví dụ: khóa bộ phận của **PAYMENT** là **payment-number**, vì trong phạm vi **một khoản nợ**, mỗi mã số trả tiền xác định duy nhất một lần trả.

Thực thể yếu – khóa chính

- Khóa chính của tập thực thể yếu = **khóa chính của tập thực thể định danh** ∪ **khóa bộ phận**.
- Với ví dụ trên: khóa chính của **PAYMENT** là **(loan-number, payment-number)**, trong đó **loan-number** là khóa chính của tập thực thể định danh **LOAN**, còn **payment-number** là khóa bộ phận của **PAYMENT**.



PHẦN 4. MÔ HÌNH ER MỞ RỘNG

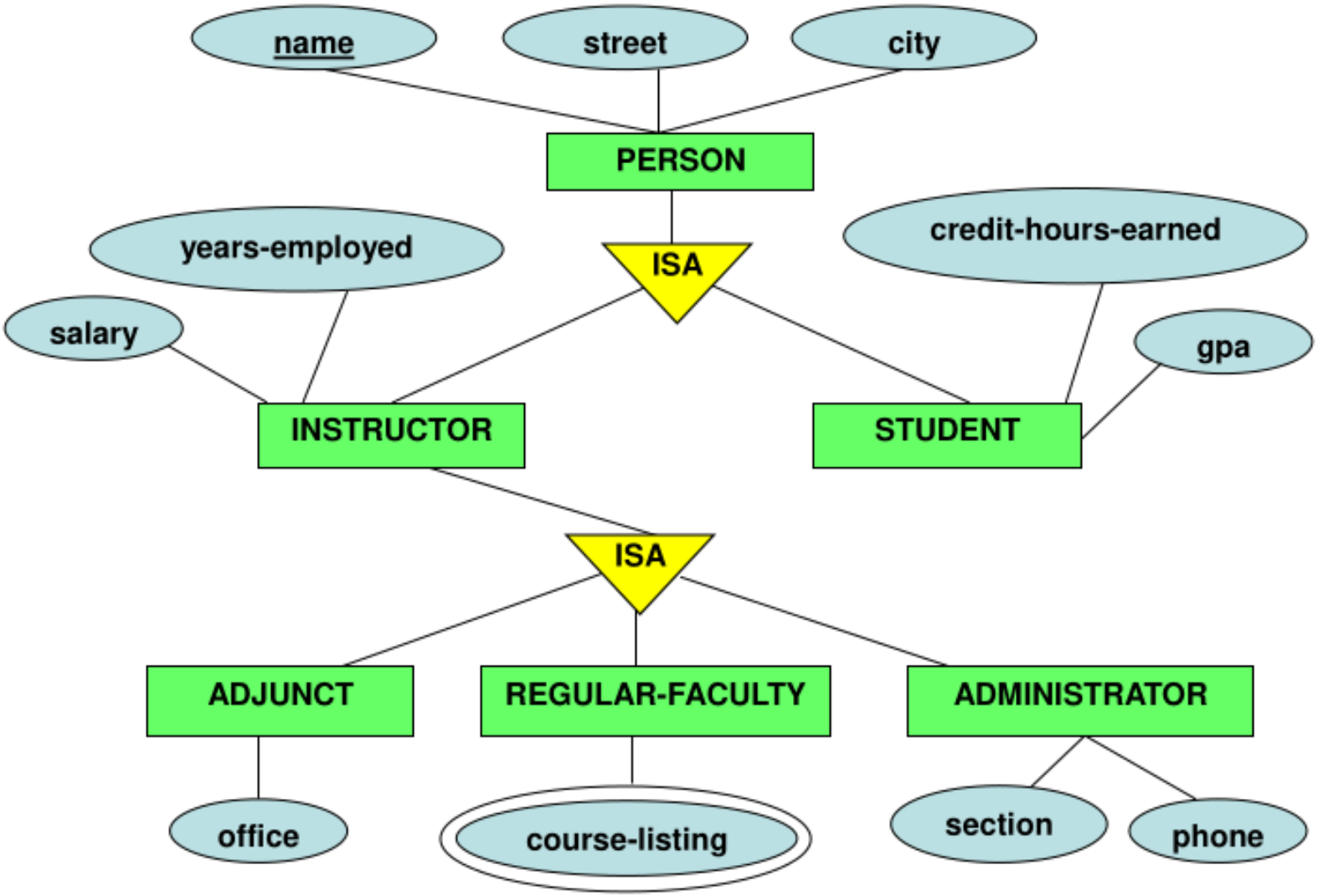
Specialization, Generalization, Inheritance

Cụ thể hóa (Specialization)

- Cụ thể hóa là quá trình **phân nhóm** bên trong một tập thực thể: tách ra các tập thực thể con khi chúng có thêm những thuộc tính riêng không giống nhau.
- Ví dụ: `PERSON (name, address, city)`

  Một **PERSON** có thể là **STUDENT** (sinh viên) hoặc **INSTRUCTOR** (giảng viên). Mỗi tập con được mô tả bởi **toàn bộ thuộc tính của PERSON** cộng thêm thuộc tính riêng:
  - **STUDENT** bổ sung **gpa**, **credit-hours-earned**;
  - **INSTRUCTOR** bổ sung **salary**, **years-employed**.
- Ký pháp ERD: tam giác **ISA** nối tập thực thể cha xuống các tập thực thể con.

Cụ thể hóa – ví dụ nhiều mức



Tổng quát hóa (Generalization)

- Cụ thể hóa là cách tiếp cận **từ trên xuống** (top-down). Quá trình ngược lại, **từ dưới lên** (bottom-up), gọi là tổng quát hóa.

  $\Longrightarrow$ Tổng quát hóa **gộp** nhiều tập thực thể thành một tập thực thể ở mức cao hơn, dựa trên các thuộc tính chung của chúng.

Tổng quát hóa – ví dụ

- Xét ví dụ trước: **PERSON** là tập thực thể mức cao (cha); **STUDENT** và **INSTRUCTOR** là các tập thực thể mức thấp (con).
  - Bước 1 – xuất phát từ hai tập thực thể độc lập:
    `STUDENT (name, address, city, gpa, credit-hours-earned)`
    `INSTRUCTOR (name, address, city, salary, years-employed)`
  - Bước 2 – tìm các thuộc tính chung để tạo tập thực thể mức cao:
    `PERSON (name, address, city)`

Cụ thể hóa và tổng quát hóa – giống và khác

- **Giống nhau**: hai quá trình cho ra **cùng một ký pháp** trên lược đồ ER (tam giác ISA). Nhìn vào một ERD đã hoàn thành, không thể biết nó được tạo ra theo hướng nào.
- **Khác nhau**: chúng là hai **quy trình thiết kế** ngược chiều, khác nhau ở điểm xuất phát và mục đích:
  - Cụ thể hóa xuất phát từ **một** tập thực thể; nhấn mạnh **sự khác nhau** giữa các thực thể bằng cách tạo ra các tập con ở mức thấp hơn.
  - Tổng quát hóa xuất phát từ **nhiều** tập thực thể có đặc tính chung; nhấn mạnh **tính tương đồng**, gộp chúng lên một tập ở mức cao hơn và giấu đi khác biệt.

Kế thừa thuộc tính

- Tập thực thể mức thấp **kế thừa mọi thuộc tính** của tập thực thể mức cao.

  Ví dụ: **STUDENT** và **INSTRUCTOR** đều kế thừa *name*, *address*, *city* của **PERSON**.
- Tập thực thể mức thấp cũng **kế thừa các liên kết** mà tập thực thể mức cao tham gia.
- Cây phân cấp các tập thực thể:
  - Tập thực thể tổng quát nhất nằm ở đỉnh cây.
  - Kế thừa đơn: tập thực thể mức thấp tham gia **đúng một** liên kết ISA.
  - Đa kế thừa: tập thực thể mức thấp tham gia **nhiều hơn một** liên kết ISA (khi đó cấu trúc là một *lưới*, không còn là cây).

Ràng buộc 1: điều kiện thành viên

- **Ràng buộc thứ nhất** xác định **thực thể nào** được là thành viên của tập thực thể mức thấp. Có 2 cách:
  - Theo mệnh đề xác định (condition-defined): thành viên được quyết định **tự động** bằng cách kiểm tra một điều kiện tường minh trên thuộc tính.
    - Nếu `job_type = 'Instructor'` thì thực thể `PERSON` đó thuộc `INSTRUCTOR`.
    - Nếu `job_type = 'Student'` thì thực thể đó thuộc `STUDENT`.
      Khi mọi tập con dùng chung một thuộc tính để phân loại, ta gọi đó là cụ thể hóa theo thuộc tính (attribute-defined).
  - Do người dùng xác định (user-defined): **không** có điều kiện tự động nào; việc gán thực thể vào tập con do người quản trị CSDL quyết định và nhập vào thủ công.

Ràng buộc 2: giao nhau hay không giao nhau

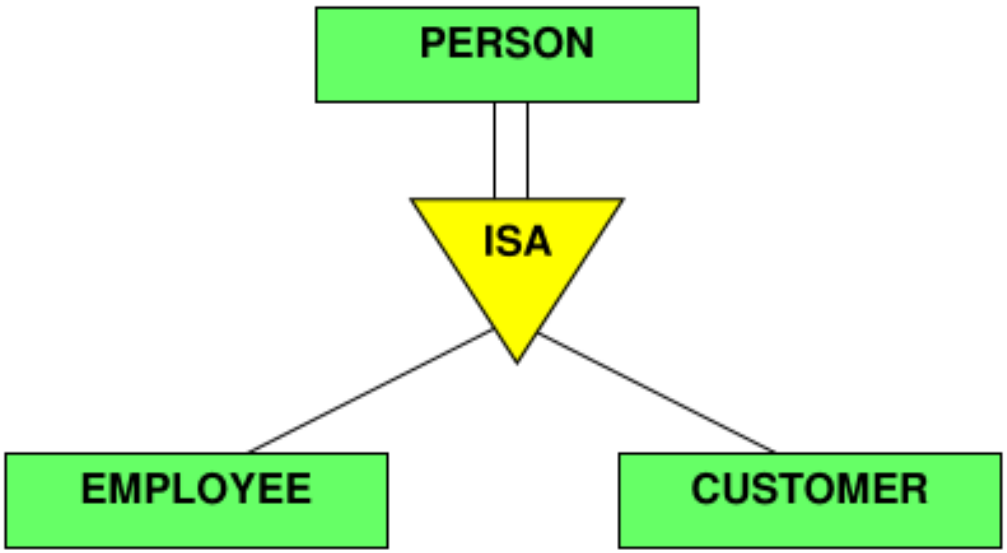
- **Ràng buộc thứ hai** xác định một thực thể mức cao có được thuộc **nhiều hơn một** tập thực thể mức thấp hay không:
  - Không giao nhau (disjoint): một thực thể chỉ được thuộc **nhiều nhất một** tập mức thấp.
    - Ví dụ: một `PERSON` chỉ có thể là `STUDENT` **hoặc** `INSTRUCTOR`, không đồng thời cả hai.
    - Ký pháp ERD: ghi chữ `disjoint` cạnh tam giác ISA.
  - Giao nhau (overlapping): một thực thể có thể thuộc **nhiều** tập mức thấp cùng lúc.
    - Ví dụ: một `PERSON` có thể vừa là `STUDENT` vừa là `INSTRUCTOR` (nghiên cứu sinh đi dạy).
    - Ký pháp ERD: ghi `overlapping`, hoặc để trống (đây là **trường hợp mặc định**).

Ràng buộc 3: toàn bộ hay một phần

- **Ràng buộc thứ ba** (ràng buộc tính toàn bộ) xác định một thực thể ở mức cao có **bắt buộc** phải thuộc vào một tập thực thể mức thấp nào đó hay không:
  - Toàn bộ (total): mọi thực thể ở mức cao **đều phải** thuộc ít nhất một **tập** thực thể mức thấp.
    - Ví dụ: mọi `PERSON` đều phải là `STUDENT` hoặc `INSTRUCTOR`.
    - Ký pháp ERD: tam giác ISA nối với tập thực thể mức cao bằng **đường đôi**.
  - Một phần (partial): tồn tại thực thể ở mức cao không thuộc tập mức thấp nào.
    - Ví dụ: một số `PERSON` không phải `STUDENT` cũng không phải `INSTRUCTOR`.
    - Ký pháp ERD: đường nối đơn. Đây là trường hợp mặc định.

Ràng buộc 2 và ràng buộc 3 **độc lập** với nhau: một cụ thể hóa có thể là toàn bộ–giao nhau, toàn bộ–không giao nhau, một phần–giao nhau hoặc một phần–không giao nhau.

Ví dụ ERD với các ràng buộc



Tổng quát hóa **toàn bộ** (đường đôi) và **có giao nhau** (không ghi `disjoint`).

Nghĩa là: mỗi người liên quan tới công ty đều phải là nhân viên hoặc khách hàng, và có thể đồng thời là cả hai.

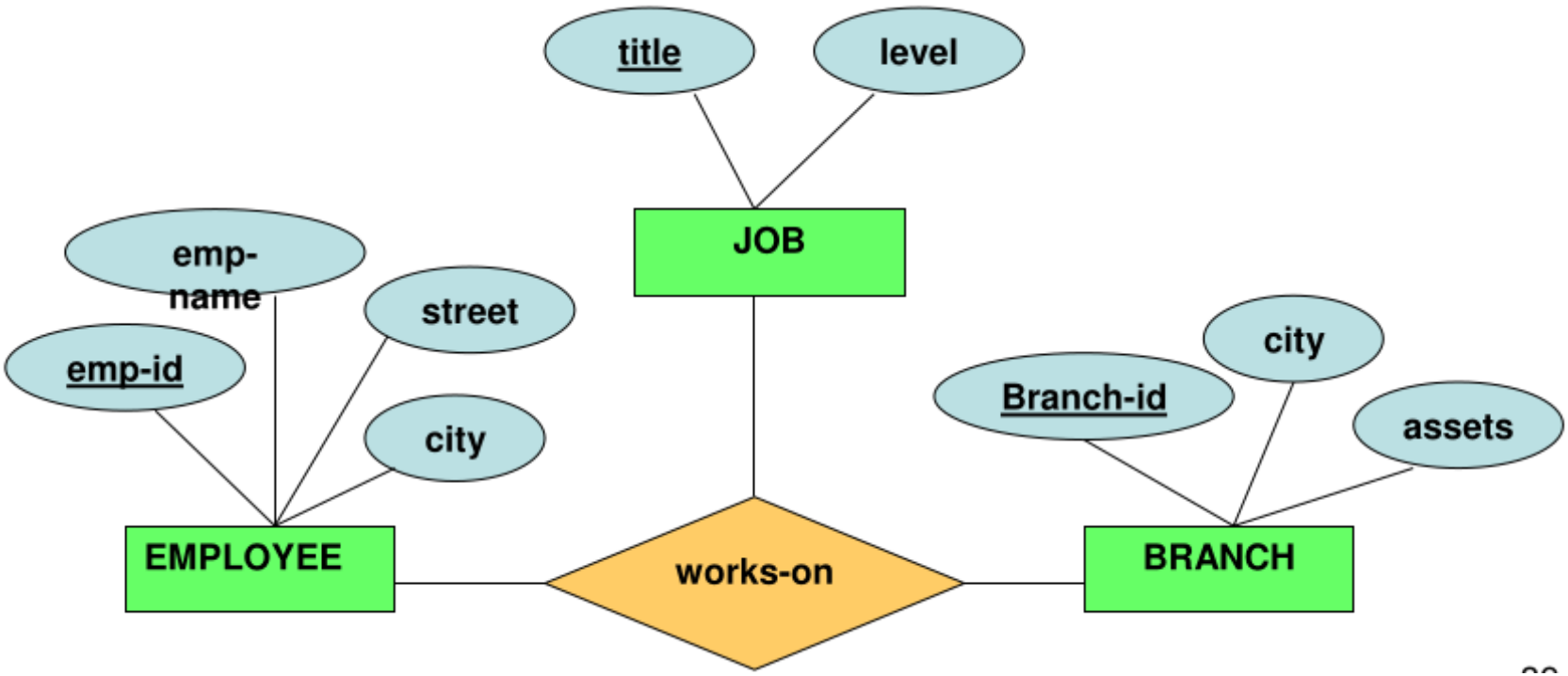
PHẦN 5. KẾT HỢP VÀ LIÊN KẾT NHIỀU NGÔI

Aggregation and n-ary Relationships

Kết hợp – vấn đề đặt ra

- Hạn chế của mô hình ER: **không thể tạo liên kết giữa một liên kết và một tập thực thể** – hình thoi chỉ được nối với hình chữ nhật.

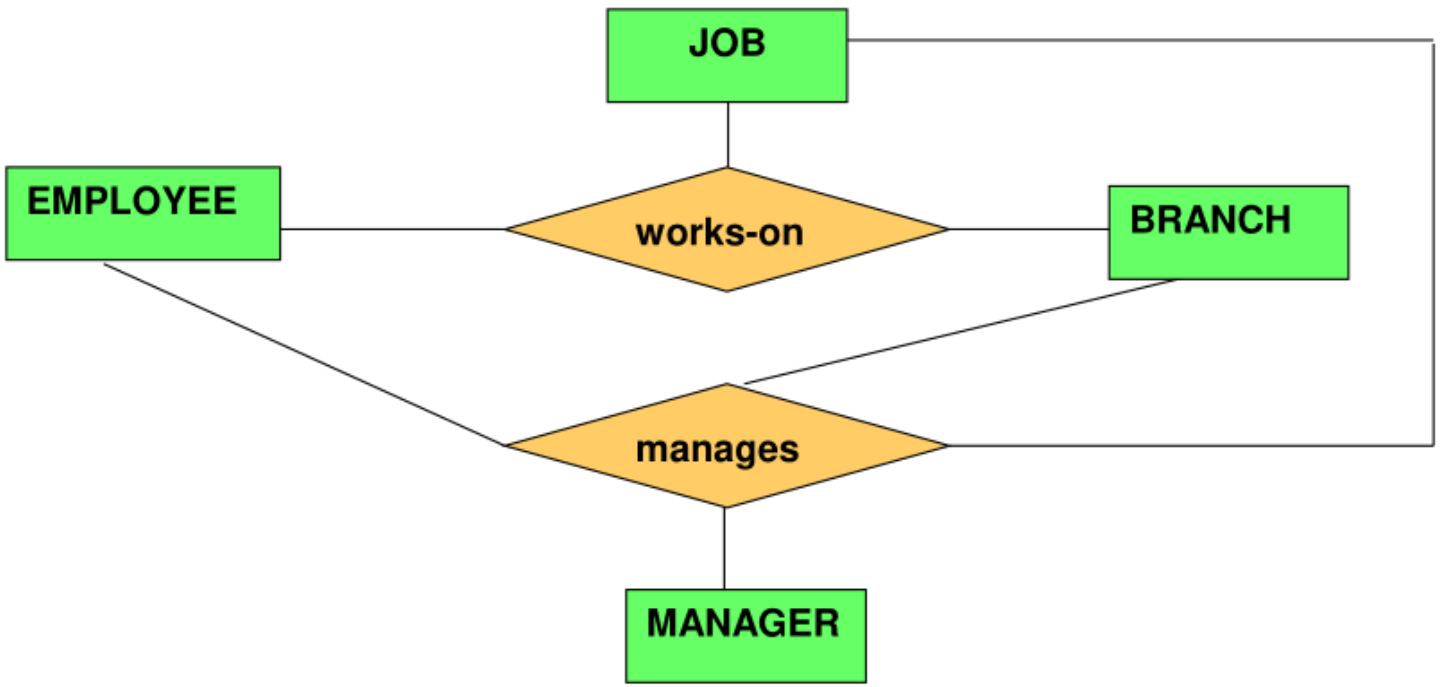
  Xét liên kết tam phân **works-on** giữa **EMPLOYEE** (*nhân viên*), **BRANCH** (*chi nhánh*) và **JOB** (*công việc*):



Kết hợp – cách làm chưa tốt

Giả sử cần ghi nhận **ai quản lý** từng công việc do một nhân viên thực hiện tại một chi nhánh, tức là lưu người quản lý cho **bộ ba** (**EMPLOYEE, BRANCH, JOB**).

$\Longrightarrow$ Một cách làm: tạo liên kết **bốn ngôi manages** nối thêm tập thực thể **MANAGER**.



Kết hợp – vì sao không dùng liên kết hai ngôi?

Câu hỏi

Tại sao không đơn giản tạo một liên kết **hai ngôi** giữa **MANAGER** và **EMPLOYEE**?

Kết hợp – vì sao không dùng liên kết hai ngôi?

Câu hỏi

Tại sao không đơn giản tạo một liên kết **hai ngôi** giữa **MANAGER** và **EMPLOYEE**?

Trả lời

Vì liên kết hai ngôi chỉ nói được "người quản lý quản lý nhân viên này", mà **không** nói được nhân viên đó được quản lý **trong công việc nào, tại chi nhánh nào**. Thông tin cần lưu gắn với cả bộ ba (EMPLOYEE, BRANCH, JOB), chứ không gắn riêng với EMPLOYEE.

Kết hợp – nhận xét về cách làm bốn ngôi

- Về hình thức, có thể nghĩ tới việc gộp **works-on** và **manages** thành một tập liên kết duy nhất.

  Tuy nhiên không gộp được, vì một bộ ba (EMPLOYEE, BRANCH, JOB) **có thể không có người quản lý** – gộp lại sẽ làm mất các bộ ba đó.
- Giữ nguyên hai tập liên kết thì lại **dư thừa**: mọi bộ ba xuất hiện trong **manages** đều đã có trong **works-on**.
- Nếu coi người quản lý là một **giá trị** chứ không phải thực thể, có thể biến **MANAGER** thành thuộc tính đa trị của **works-on**.

  $\Longrightarrow$ Cách này làm việc truy vấn khó hơn cả về logic lẫn chi phí (ví dụ: bộ ba *employee–branch–job* do người quản lý nào chịu trách nhiệm?).

Kết hợp – giải pháp

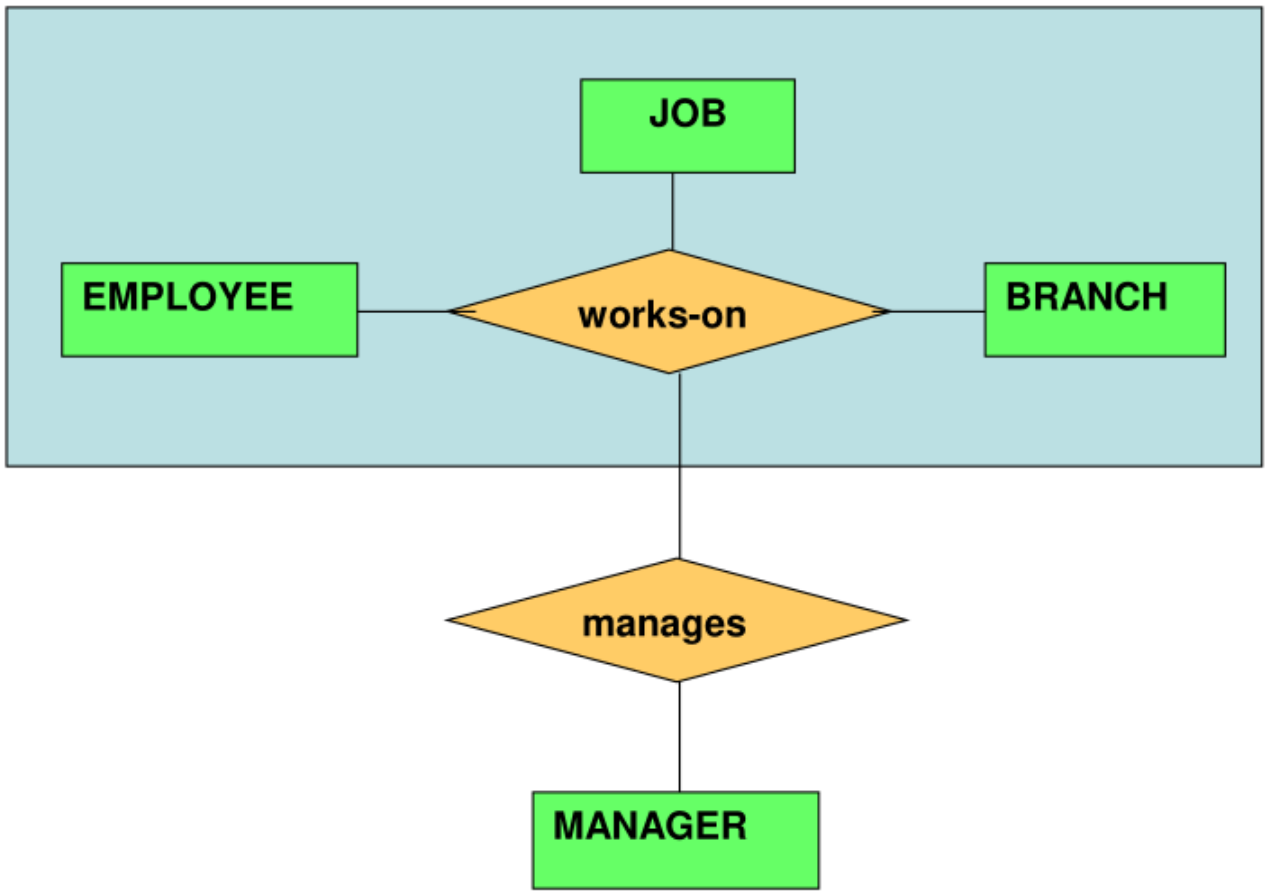
$\Longrightarrow$ Cách tốt nhất là dùng **kết hợp** (aggregation). Kết hợp là một phép trừu tượng, coi **một tập liên kết** như **một tập thực thể ở mức cao hơn**.

$\Longrightarrow$ **Áp dụng cho ví dụ trên**:

- Coi tập liên kết **works-on** (nối **EMPLOYEE**, **BRANCH**, **JOB**) như một tập thực thể mức cao, đặt tên **WORKS-ON**.
- Tạo liên kết **hai ngôi manages** giữa **WORKS-ON** và **MANAGER**.

- Ký pháp ERD: bao phần được kết hợp trong một **hình chữ nhật lớn**.

Kết hợp trong mô hình ER



Cách này vừa giữ được các bộ ba không có người quản lý, vừa không lặp lại dữ liệu.

Liên kết nhiều ngôi

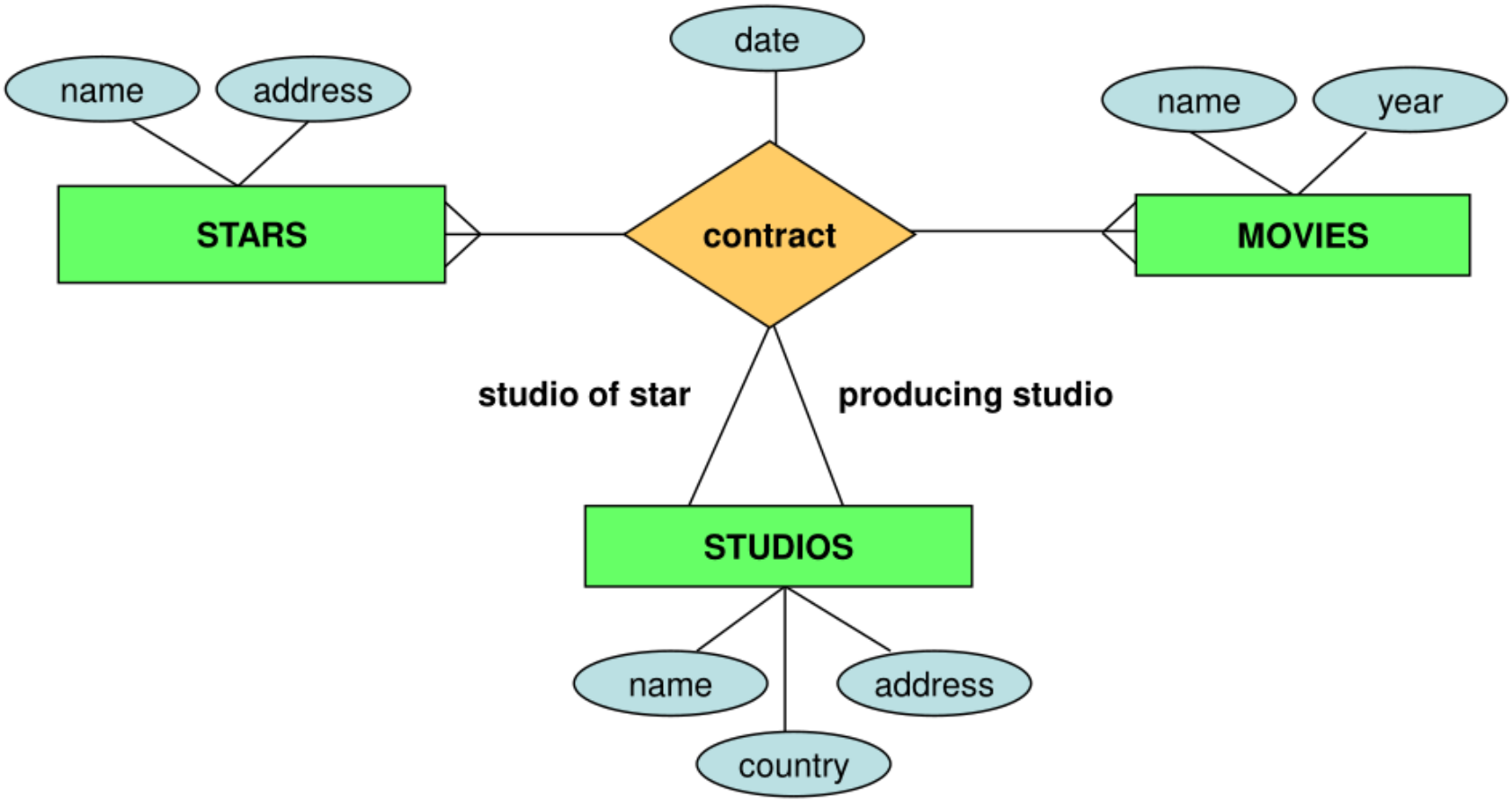
- Hầu hết liên kết đã xét là **nhị phân** (nối hai tập thực thể).
- Một liên kết **n ngôi** ($n > 2$) luôn có thể được thay bằng một **tập thực thể mới** E cùng n liên kết nhị phân dạng **nhiều–một** từ E tới từng tập thực thể tham gia.

  $\Longrightarrow$ Điều này hữu ích vì mô hình ER cho phép liên kết nhiều ngôi, nhưng nhiều mô hình dữ liệu khác (và các công cụ thiết kế) chỉ hỗ trợ liên kết nhị phân.

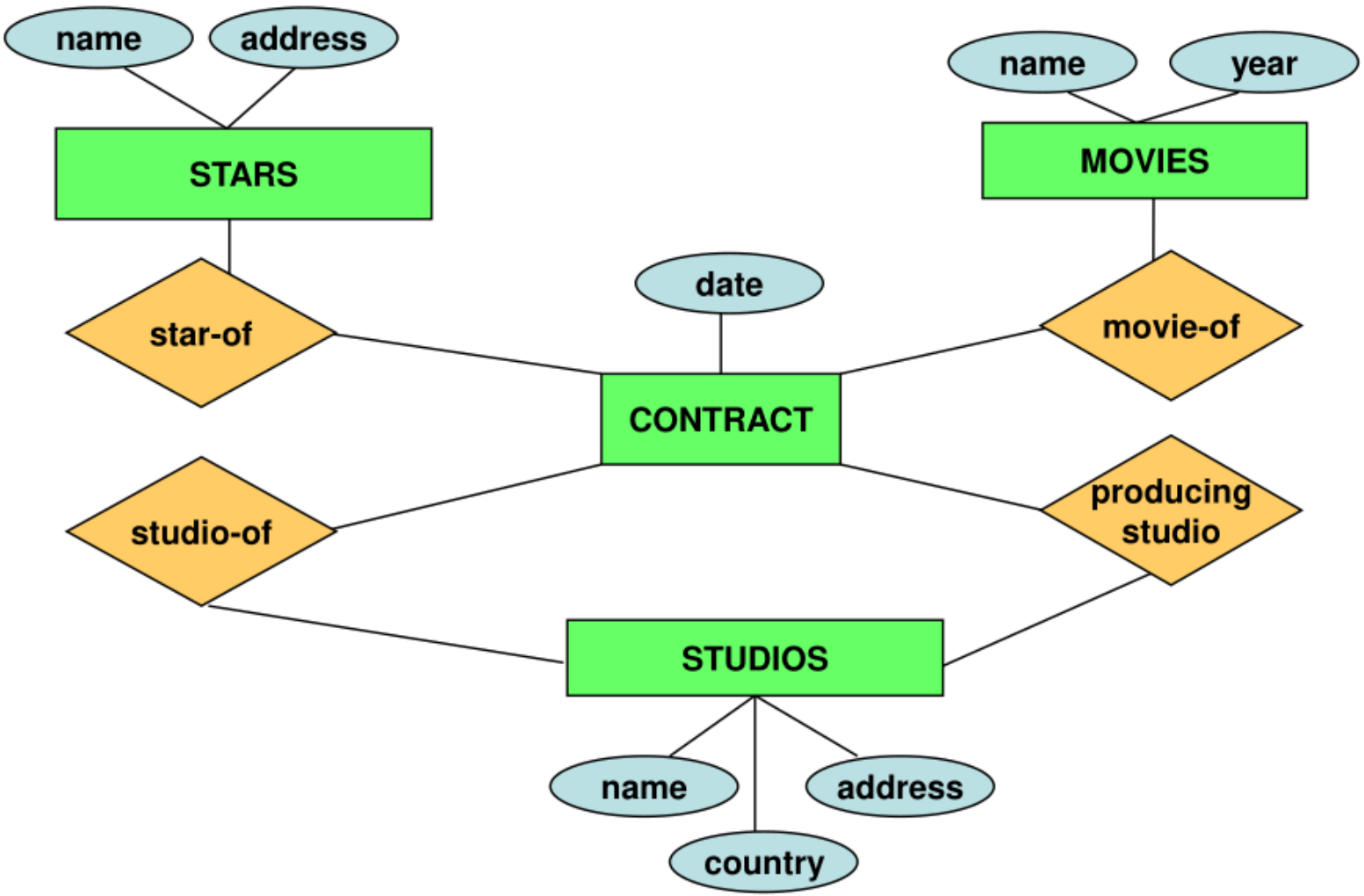
  Lưu ý: phép biến đổi này **không tự động bảo toàn ngữ nghĩa**. Các ràng buộc của liên kết n ngôi gốc phải được phát biểu bổ sung, vì bản thân lược đồ nhị phân không diễn đạt được chúng.

Liên kết nhiều ngôi – ví dụ

Liên kết tam phân **contract** giữa STARS, MOVIES và STUDIOS:

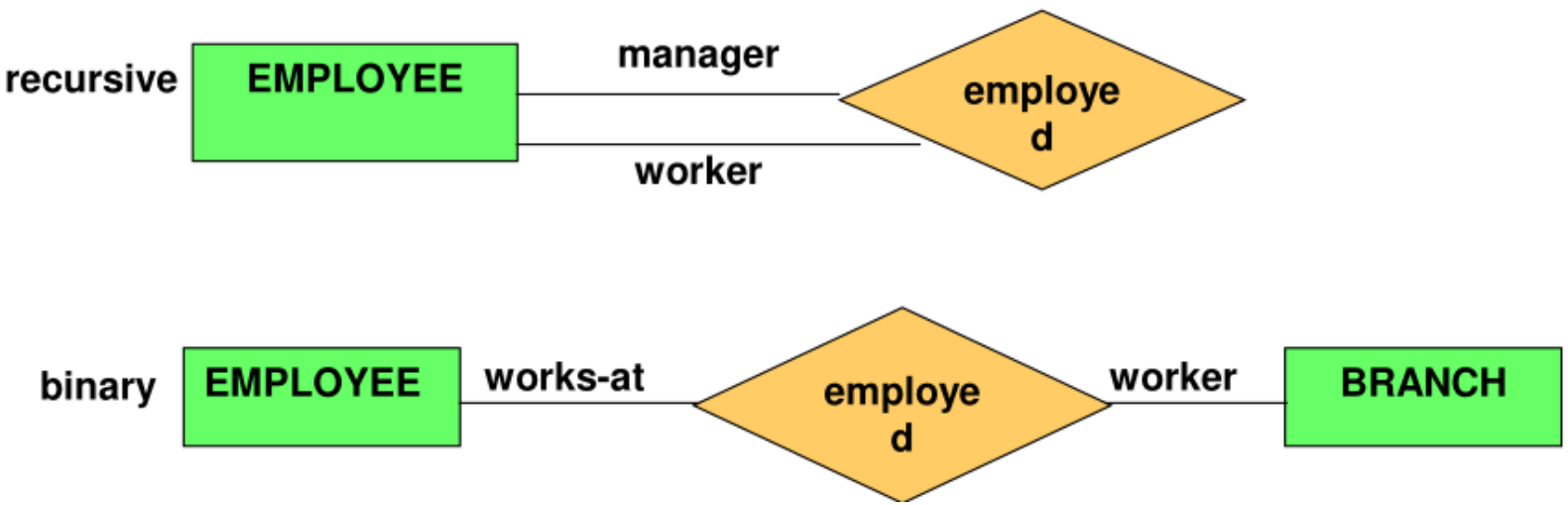


Chuyển liên kết nhiều ngôi thành các liên kết nhị phân



Lược đồ ER với chỉ thị vai trò

- **Vai trò** (role) được biểu diễn bằng cách gán nhãn lên đường nối giữa tập thực thể và tập liên kết.
- Vai trò **bắt buộc** phải ghi với liên kết **đệ quy**, và **nên** ghi với liên kết nhị phân hoặc nhiều ngôi khi tên liên kết chưa đủ rõ nghĩa.



PHẦN 6. ER VÀ UML

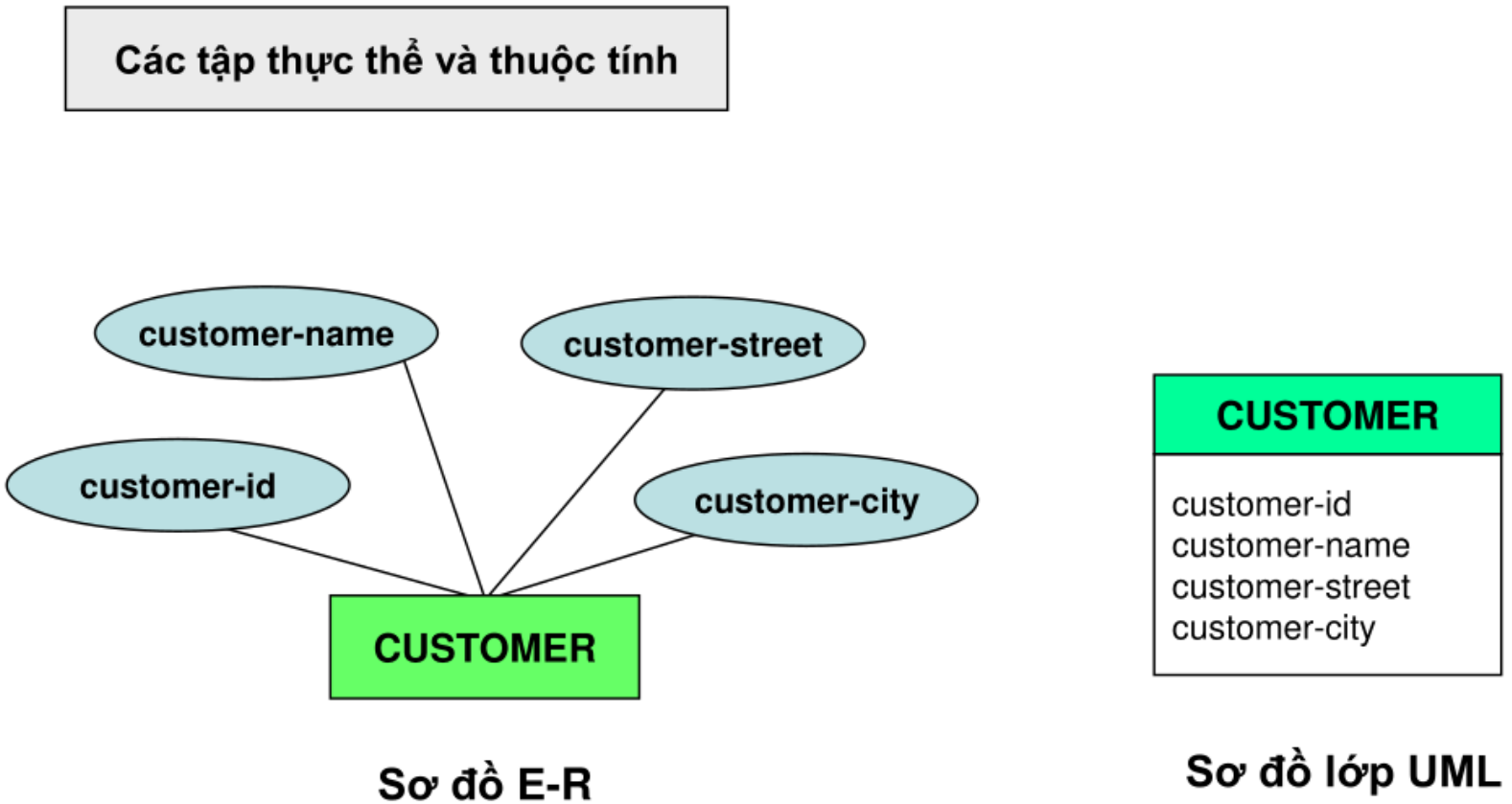
ER Diagram vs. UML Class Diagram

Ngôn ngữ UML (Unified Modeling Language)

Một số thành phần của UML:

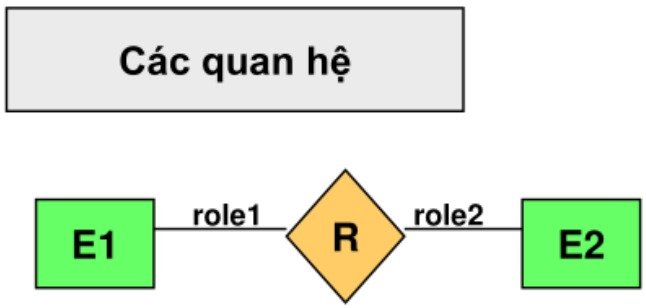
1. **Sơ đồ lớp** (class diagram): tương ứng gần nhất với lược đồ ER – đây là phần ta sẽ đối chiếu.
2. **Sơ đồ use case**: thể hiện tương tác giữa người dùng và hệ thống (ví dụ: rút tiền từ tài khoản, đăng ký một khóa học).
3. **Sơ đồ hoạt động**: thể hiện luồng công việc giữa các thành phần trong hệ thống.
4. **Sơ đồ cài đặt**: thể hiện các thành phần của hệ thống và kết nối giữa chúng, ở cả khía cạnh phần mềm lẫn phần cứng.

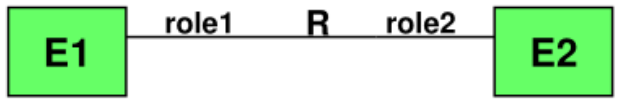
ER và UML: tập thực thể và thuộc tính

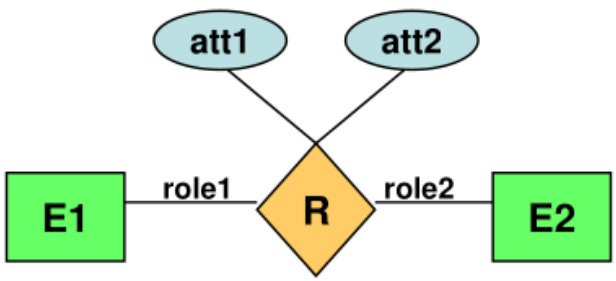


Thuộc tính trong ER vẽ thành elip riêng; trong UML được liệt kê bên trong hộp lớp.

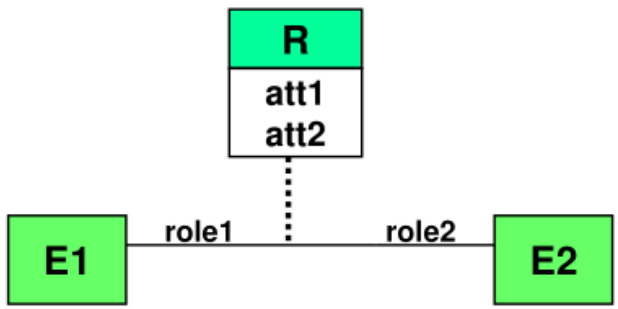
ER và UML: liên kết







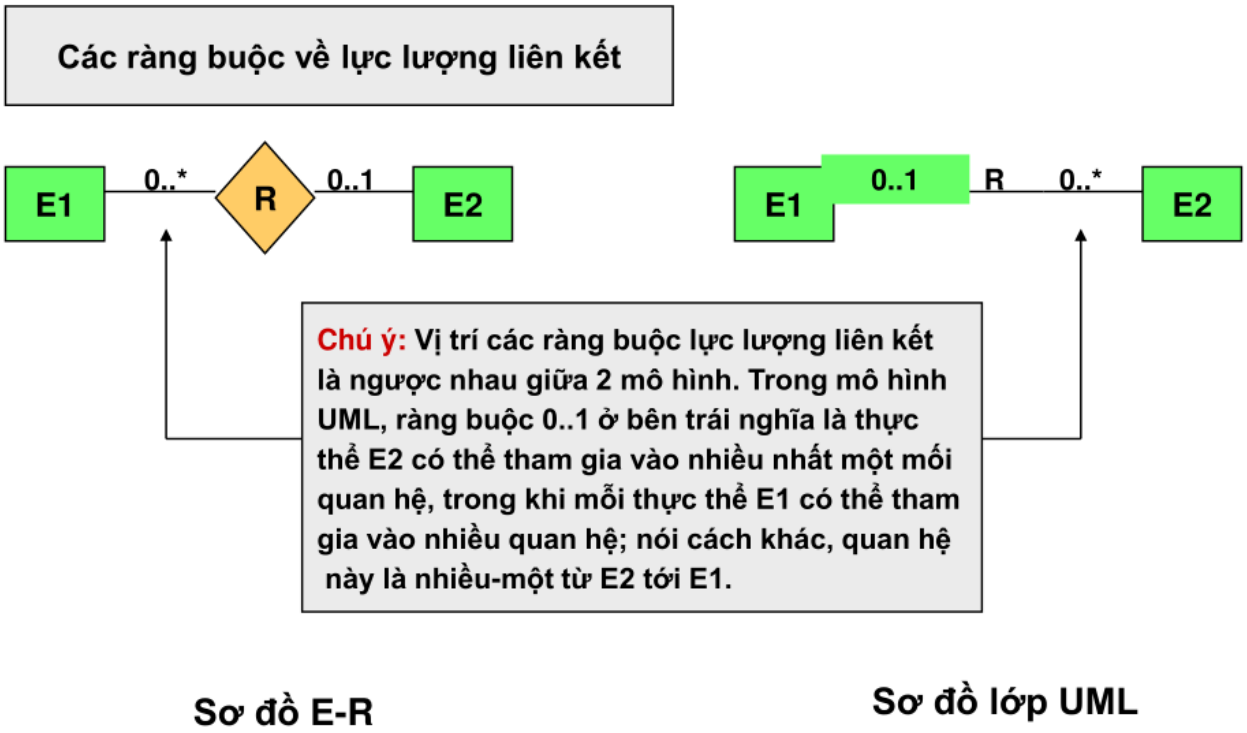
Sơ đồ E-R



Sơ đồ lớp UML

Liên kết có thuộc tính riêng được UML biểu diễn bằng *lớp liên kết* (association class), nối bằng nét đứt.

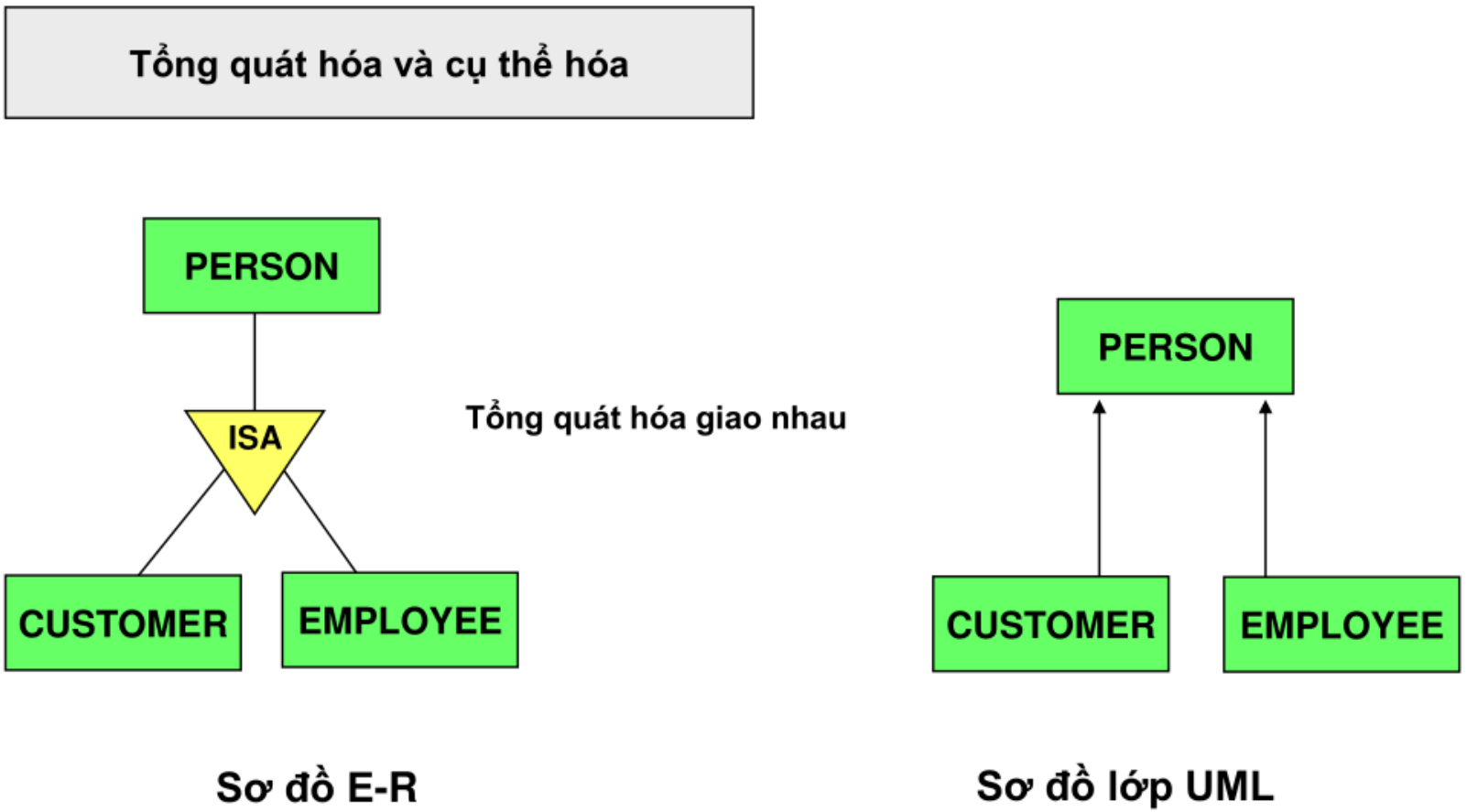
ER và UML: ràng buộc lực lượng – điểm rất dễ nhầm



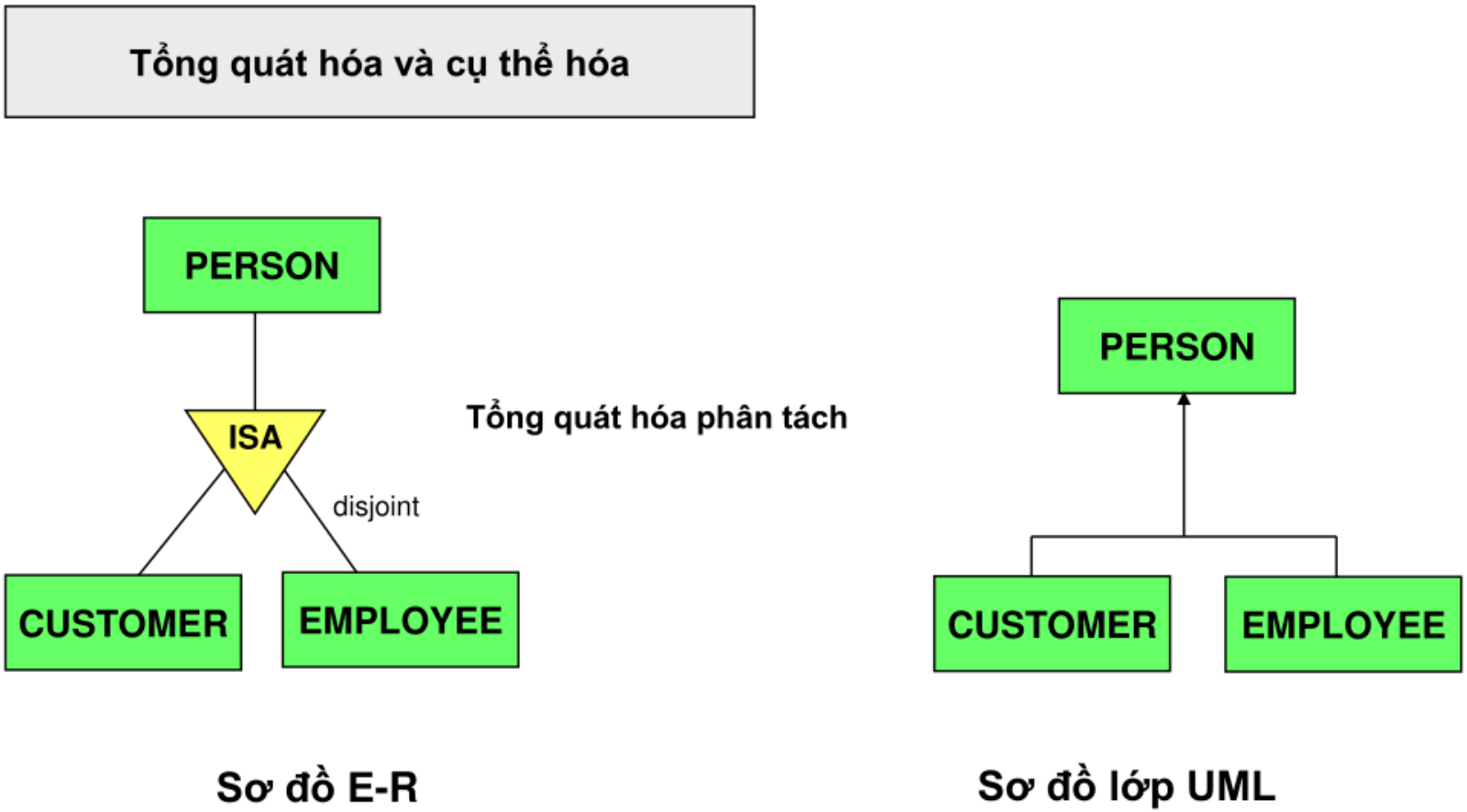
Chú ý

Vị trí ghi ràng buộc lực lượng của hai mô hình là **ngược nhau**. Trong UML, ký hiệu `0..1` ghi ở **bên trái** (phía E_1) mô tả ràng buộc của E_2; nói cách khác đây là liên kết **nhiều–một từ** E_2 **tới** E_1.

ER và UML: tổng quát hóa giao nhau



ER và UML: tổng quát hóa không giao nhau



ER ghi chữ `disjoint`; UML gộp các mũi tên kế thừa vào một nhánh chung.

PHẦN 7. RÀNG BUỘC TOÀN VẸN THAM CHIẾU

Referential Integrity

Ràng buộc toàn vẹn tham chiếu

- Ở mức ER: mọi thực thể **được tham chiếu tới** bởi một liên kết đều **phải tồn tại** trong CSDL.
- Ở mức quan hệ (Chương 3): giá trị xuất hiện ở thuộc tính **khóa ngoại** của một bảng phải **tồn tại** trong tập giá trị **khóa chính** của bảng được tham chiếu (hoặc nhận giá trị NULL).

  $\Longrightarrow$ Ngăn chặn dữ liệu "mồ côi" (orphan records) – dữ liệu trỏ tới một đối tượng không tồn tại.
- Các phương án xử lý khi thao tác xóa/sửa làm vi phạm ràng buộc:
  - *No Action / Restrict*: từ chối thao tác nếu vẫn còn dữ liệu tham chiếu.
  - *Cascade*: xóa/sửa dây chuyền sang các bảng tham chiếu.
  - *Set Null* hoặc *Set Default*: gán NULL hoặc giá trị mặc định cho khóa ngoại.

Ví dụ minh họa: ràng buộc tham chiếu

Bảng LOP (bảng cha)

MaLop	**TenLop**
L01	CNTT 1
L02	An toàn TT

MaLop là **khóa chính**.

Bảng SINH_VIEN (bảng con)

MaSV	**HoTen**	**MaLop**
SV01	Nguyễn Văn A	L01
SV02	Trần Thị B	L02
SV03	Lê Văn C	L09

MaLop là **khóa ngoại**, tham chiếu tới LOP.

Phân tích lỗi

Dòng **SV03** vi phạm ràng buộc tham chiếu vì mã lớp **L09** không tồn tại trong bảng **LOP**. Hệ quản trị CSDL sẽ từ chối thêm dòng này.

PHẦN 8. TỔNG KẾT

Nguyên tắc thiết kế, tóm tắt và bài tập

Các nguyên tắc thiết kế mức khái niệm

- Nguyên tắc chung:
  - Khảo sát kỹ quy trình nghiệp vụ của người dùng và ứng dụng.
  - Trao đổi với nhiều nhóm người dùng khác nhau.
  - Dùng thuật ngữ của thế giới thực, đúng như người dùng gọi.
  - Chính xác, đơn giản, dễ hiểu và tránh trùng lặp.
- Các thành phần cần nhận dạng khi xây dựng lược đồ:
  - Các kiểu thực thể, kiểu liên kết và thuộc tính.
  - Thuộc tính khóa, tỉ lệ lực lượng và ràng buộc tham gia.
  - Các kiểu thực thể yếu.
  - Cụ thể hóa và tổng quát hóa.
- Hai hướng tiếp cận:
  - Thiết kế trực tiếp lược đồ tổng thể.
  - Thiết kế các lược đồ con rồi **tích hợp** thành lược đồ tổng thể.

Các câu hỏi cần trả lời khi tinh chỉnh lược đồ

- Khái niệm này nên là **kiểu thực thể** hay **thuộc tính**?
- Khái niệm này nên là **kiểu liên kết** hay **thuộc tính**?
- Thuộc tính của liên kết nên **đặt ở đâu** – trên liên kết hay trên tập thực thể?
- Tập thực thể này là **mạnh** hay **yếu**?
- Có cần **cụ thể hóa** / **tổng quát hóa** không? Ràng buộc đi kèm là gì?
- Nên dùng **liên kết nhị phân** hay **liên kết n ngôi**? Có cần **kết hợp** không?

Tóm tắt bài học

- Lược đồ tập liên kết luôn chứa hợp các khóa chính; **tỉ lệ lực lượng** chỉ quyết định **khóa chính** là gì.
- Thuộc tính chỉ chuyển được từ liên kết sang tập thực thể khi thực thể đó tham gia **nhiều nhất một** liên kết; liên kết **M:N** thì không chuyển được.
- Chọn thực thể / thuộc tính / liên kết là quyết định dựa trên **ngữ nghĩa nghiệp vụ**, ưu tiên phương án tránh dư thừa.
- Thực thể yếu: khóa chính = khóa thực thể định danh $\cup$ khóa bộ phận.
- EER bổ sung ISA với **3 ràng buộc**: điều kiện thành viên, giao nhau/không giao nhau, toàn bộ/một phần.
- **Kết hợp** coi một tập liên kết như một tập thực thể mức cao, để tạo được liên kết giữa các liên kết.

Bài tập

1. Cho liên kết **WORKS-FOR** giữa NHANVIEN và PHONGBAN với thuộc tính *ngày_bắt_đầu*. Xác định khóa chính của tập liên kết và vị trí hợp lý của thuộc tính trong ba trường hợp: 1:1, 1:N, M:N.
2. Trong hệ thống quản lý khách sạn, "phòng" nên là thực thể hay thuộc tính của "khách sạn"? Nêu tiêu chí và lập luận.
3. Cho tập thực thể `CHUONG(so_chuong, tieu_de)` trong hệ thống quản lý sách. Đây là thực thể mạnh hay yếu? Xác định khóa bộ phận và khóa chính.
4. Vẽ lược đồ EER cho tập thực thể NGUOI trong một trường đại học với các tập con SINHVIEN, GIANGVIEN, NHANVIEN_HANHCHINH. Xác định và ghi rõ cả ba loại ràng buộc.
5. Xét liên kết tam phân **CUNGCAP**(NHACUNGCAP, VATTU, DUAN). Nếu cần lưu thêm người phê duyệt cho từng bộ ba, hãy mô hình hóa bằng **kết hợp** và giải thích vì sao không dùng liên kết bốn ngôi.